Composite

Composite · Compute total size of nested project folders · Build with Waqar

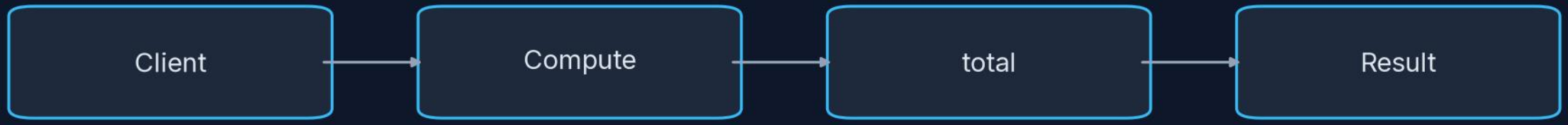
Today's Hook

- Problem: Compute total size of nested project folders
- Pattern: Composite — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Compute total size of nested project folders
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Composite: Compute total size of nested project folders

Real-World Scenario

- Running example: Compute total size of nested project folders
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.gof.structural.composite;
import com.buildwithwaqar.patterns.gof.structural.composite.*;
/** Problem: total size of nested project directories. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Composite problem: folder size ===");
    var root = new DirectoryComposite("repo")
      .add(new FileLeaf("README.md", 100))
      .add(new DirectoryComposite("src").add(new FileLeaf("Main.java", 500)).add(new FileLeaf("Demo.java", 200)));
    System.out.println("totalBytes=" + root.size());
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.gof.structural.composite;

public sealed interface FileSystemNode permits FileLeaf, DirectoryComposite {
    String name();
    int size();
}


package com.buildwithwaqar.patterns.gof.structural.composite;

import java.util.ArrayList;
import java.util.List;

public final class DirectoryComposite implements FileSystemNode {
    private final String name;
    private final List<FileSystemNode> children = new ArrayList<>();

    public DirectoryComposite(String name) { this.name = name; }

    public DirectoryComposite add(FileSystemNode node) {
        children.add(node);
        return this;
    }
```

Solution (2)

```
    @Override public String name() { return name; }

    @Override public int size() {
        return children.stream().mapToInt(FileSystemNode::size).sum();
    }
}
```

Runnable main

```java
package com.buildwithwaqar.patterns.gof.structural.composite;
import com.buildwithwaqar.patterns.gof.structural.composite.*;
/** Problem: total size of nested project directories. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Composite problem: folder size ===");
    var root = new DirectoryComposite("repo")
      .add(new FileLeaf("README.md", 100))
      .add(new DirectoryComposite("src").add(new FileLeaf("Main.java", 500)).add(new FileLeaf("Demo.java", 200)));
    System.out.println("totalBytes=" + root.size());
  }
}
```

Walkthrough Output

```
=== Composite problem: folder size ===
totalBytes=800
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Composite.

Related Patterns

- Contrast Composite with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Composite → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.